

Prefață

Acest curs a fost conceput pentru a ghida un student autodidact de la primele noțiuni despre programare până la nivelul în care poate aborda proiecte complexe scrise în C++. Structura materialului urmărește o progresie logică: de la bazele limbajului și ale paradigmatelor fundamentale, trecând prin programarea orientată pe obiecte, programarea generică și utilizarea bibliotecii standard, până la instrumente moderne introduse în standardele recente. Fiecare modul include:

- **Teorie** cu explicații detaliate și context istoric acolo unde este relevant.
- **Exemple de cod** anotate pentru a evidenția cele mai importante concepte.
- **Exerciții** cu niveluri diferite de dificultate, utile pentru consolidarea cunoștințelor.
- **Proiecte** și provocări pentru a combina mai multe concepte într-o aplicație coerentă.

> **Cum să folosești cursul:** parcurge fiecare modul în ordine, implementează exemplele manual și rezolvă exercițiile înainte de a verifica soluțiile propuse. Utilizează capitolul [Resurse](#) pentru a aprofunda subiectele preferate și revino asupra modulelor unde simți că mai ai nevoie de clarificări.

Modulul 1. Introducere în C++

Obiective

- Înțelegerea istoricului și a filosofiei din spatele limbajului C++.
- Configurarea mediului de dezvoltare pe diferite platforme.
- Scrierea și compilarea primului program.

Teorie

C++ a fost creat de Bjarne Stroustrup la începutul anilor 1980 ca o extensie a limbajului C pentru programarea orientată pe obiecte. Evoluția limbajului a fost influențată de nevoia de eficiență, control explicit al resurselor și compatibilitatea cu C. Standardele majore (C++98, C++11, C++14, C++17, C++20, C++23) au adus îmbunătățiri semnificative, de la abloane și STL la programare concurentă și conceptul de corutine.

Exemplu

```
#include <iostream>

int main() {
    std::cout << "Salut, lume!" << std::endl;
    return 0;
}
```

Explicații

- â€¢ '#include <iostream>' aduce în program declarațiile pentru fluxurile de intrare/ieșire.
- â€¢ 'std::cout' este fluxul standard de ieșire.
- â€¢ 'std::endl' golește bufferul de ieșire și inserează un caracter de linie nouă.

Exerciții

1. Instalează un compilator C++ (GCC, Clang sau MSVC) și configurează un IDE sau un editor text cu suport pentru C++.
2. Scrie un program care afișează numele, ora și hobby-urile tale pe linii separate.
3. Cercetează diferențele dintre standardele C++98 și C++11 și prezintă-le într-un fișier text.

Modulul 2. Bazele limbajului

Obiective

- â€¢ Declararea variabilelor și folosirea tipurilor fundamentale.
- â€¢ Utilizarea operatorilor aritmetici, logici și de comparație.
- â€¢ Alegerea fluxurilor de intrare/ieșire.

Teorie

Tipurile de bază includ 'int', 'double', 'char', 'bool' și 'std::string'. Conversiile implicite pot duce la erori subtile, astfel că folosirea inițializărilor cu acolade '{}' este recomandată pentru a evita conversiile năștite. Literalelor li se pot adăuga sufixe ('u', 'l', 'f') pentru a specifica tipul.

Exemplu: citirea și afișarea datelor

```
#include <iostream>
#include <string>

int main() {
    std::string nume;
    int varsta;

    std::cout << "Introdu numele: ";
    std::getline(std::cin, nume);

    std::cout << "Introdu vârsta: ";
    std::cin >> varsta;

    std::cout << "Salut, " << nume << "! Ai " << varsta << " ani." << std::endl;
}
```

Exerciții

1. Creează un convertor de temperatură Celsius în Fahrenheit.
2. Implementează un program care calculează aria și perimetrul unui dreptunghi pe baza intrucerilor utilizatorului.
3. Scrie un program care determină dacă un număr întreg este par sau impar folosind operatorul '%'.

Modulul 3. Controlul fluxului

Obiective

• Folosirea instrucțiunilor 'if', 'switch', 'for', 'while', 'do-while'.

• Alegerea structurii 'break' și 'continue'.

• Aplicarea operatorului condițional '?:' și a instrucțiunii 'goto' (doar pentru referință istorică).

Teorie

Controlul fluxului permite ramificarea execuției. 'switch' este util pentru comparații discrete, dar necesită constante integrale sau enum-uri. Bucla 'for' este ideală pentru iterații cu contor cunoscut, în timp ce 'while' se potrivește iterațiilor controlate de condiții.

Exemplu: tabel de înmulțire

```
#include <iostream>

int main() {
    for (int i = 1; i <= 10; ++i) {
        for (int j = 1; j <= 10; ++j) {
            std::cout << i * j << '\t';
        }
        std::cout << '\n';
    }
}
```

Exerciții

1. Implementează un calculator de note care convertește punctajul numeric în litere (A-F) folosind 'if' și 'switch'.
2. Scrie un program care verifică dacă un număr este prim.
3. Creează un joc simplu de ghicit numere (1-100) cu buclă 'do-while' și feedback în preamă și prea mare.

Modulul 4. Funcții și modularizare

Obiective

- Definirea funcțiilor cu și fără valori de retur.
- Argumente implicite, supraîncărcare și inline.
- Separarea codului în fișiere antet (.hpp) și implementare (.cpp).

Teorie

Funcțiile sunt blocuri reutilizabile. Declarațiile se plasează, de regulă, în fișiere header pentru a permite altor module să le cunoască semnăturile. 'inline' sugerează compilatorului să înclouiască apelul cu corpul funcției, dar decizia finală aparține compilatorului. 'constexpr' permite evaluarea la compilare.

Exemplu: separarea în fișiere

```
'math_utils.hpp'
#pragma once

int aduna(int a, int b);

'math_utils.cpp'
#include "math_utils.hpp"

int aduna(int a, int b) {
    return a + b;
}

'main.cpp'
#include <iostream>
#include "math_utils.hpp"

int main() {
    std::cout << aduna(3, 4) << '\n';
}
```

Exerciții

1. Scrie funcții separate pentru calculul factorialului iterativ și recursiv; compară performanța folosind un număr de apeluri.
2. Implementează o funcție 'double medie(const std::vector<int>&)' și testează-o cu date random.
3. Creează un modul 'geometry' cu funcții pentru arii de triunghi, cerc, pătrat și un program care le utilizează.

Modulul 5. Programare orientată pe obiecte

Obiective

- Crearea claselor, constructorilor, destructorilor și membrilor statici.

â€¢ Aplicarea principiilor de încapsulare, moț ținere și polimorfism.

â€¢ Folosirea regulii celor trei/cinci/zero.

Teorie

În C++ clasele pot avea membri publici, protejați și privați. Constructorii inițializează starea, iar destructorii eliberează resurse. Polimorfismul dinamic funcționează prin intermediul pointerilor/referințelor către clase de bază și funcții virtuale. Interfețele pot fi emulate cu clase abstracte care conțin cel puțin o metodă virtuală pură.

Exemplu: ierarhie de forme

```
#include <iostream>
#include <memory>
#include <vector>

class Forma {
public:
    virtual ~Forma() = default;
    virtual double aria() const = 0;
    virtual void descrie() const {
        std::cout << "Sunt o formă generică.\n";
    }
};

class Cerc final : public Forma {
public:
    explicit Cerc(double r) : raza{r} {}
    double aria() const override { return 3.14159 * raza * raza; }
    void descrie() const override {
        std::cout << "Cerc cu raza " << raza << "\n";
    }
private:
    double raza;
};

class Dreptunghi : public Forma {
public:
    Dreptunghi(double l, double L) : latura_mica{l}, latura_mare{L} {}
    double aria() const override { return latura_mica * latura_mare; }
private:
    double latura_mica;
    double latura_mare;
};

int main() {
    std::vector<std::unique_ptr<Forma>> forme;
    forme.emplace_back(std::make_unique<Cerc>(5.0));
    forme.emplace_back(std::make_unique<Dreptunghi>(3.0, 4.0));
}
```

```

    for (const auto& forma : forme) {
        forma->descrie();
        std::cout << "Aria: " << forma->aria() << "\n";
    }
}

```

Exerciții

1. Implementează o clasă 'ContBancar' cu operații de depunere/retragere, validare a sumelor și log de tranzacții.
2. Creează o ierarhie pentru vehicule (clasă de bază 'Vehicul', derivate 'Masina',) și T* ('Bicicleta', 'A...')
3. Aplică principiul SOLID pentru a refactoriza o clasă în Zeus care face toate.

Modulul 6. Biblioteca Standard (STL)

Obiective

- Familiarizarea cu containerele secvențiale și asociative.
- Utilizarea algoritmilor generici (std::sort, std::accumulate, std::transform).
- Introducerea iteratorilor și a adaptorilor de container.

Teorie

STL este construit pe trei piloni: container, algoritmi și iteratoare. Containerele secvențiale (std::vector, std::list, std::deque) organizează elementele liniar, în timp ce containerele asociative (std::map, std::set) gestionează perechi cheie-valoare sau seturi. Algoritmii funcționează pe intervale definite de doi iteratori.

Exemplu: procesarea notelor

```

#include <algorithm>
#include <iostream>
#include <numeric>
#include <vector>

int main() {
    std::vector<int> note{7, 8, 9, 10, 6, 8};

    std::sort(note.begin(), note.end());
    auto medie = std::accumulate(note.begin(), note.end(), 0.0) / note.size();

    std::cout << "Note ordonate: ";
    for (int nota : note) {
        std::cout << nota << ' ';
    }
}

```

```
std::cout << "\nMedia: " << medie << '\n';
}
```

Exerciții

1. Analizează complexitatea principalelor containere STL și creează un tabel comparativ.
2. Scrie un program care citește cuvinte dintr-un fișier și construiește un `'std::map<std::string, int>'` cu frecvențe.
3. Utilizează `'std::partition'` pentru a separa numerele pare de cele impare într-un `'std::vector'`.

Modulul 7. Gestionarea memoriei

Obiective

- Utilizarea pointerilor și a referințelor.
- Alegerea smart pointerilor (`'std::unique_ptr'`, `'std::shared_ptr'`, `'std::weak_ptr'`).
- Implementarea conceptelor RAII și alocare personalizată.

Teorie

C++ oferă control total asupra memoriei, dar responsabilitatea este transferată programatorului. RAII (Resource Acquisition Is Initialization) garantează eliberarea resurselor la finalul duratei de viață. Smart pointerii sunt wrapperi siguri pentru pointerii brute și respectă RAII.

Exemplu: `'std::unique_ptr'`

```
#include <iostream>
#include <memory>

class Logger {
public:
    explicit Logger(std::string nume) : nume_(std::move(nume)) {
        std::cout << "Pornire logger: " << nume_ << '\n';
    }
    ~Logger() {
        std::cout << "Oprește logger: " << nume_ << '\n';
    }
    void log(const std::string& mesaj) {
        std::cout << '[' << nume_ << "]" << mesaj << '\n';
    }
private:
    std::string nume_;
};

int main() {
```

```

    auto logger = std::make_unique<Logger>("Aplicatie");
    logger->log("Pornit modulul de autentificare");
}

```

Exerciții

1. Converteți un cod existent care folosește 'new'/'delete' într-o variantă bazată pe smart pointeri.
2. Implementați un allocator personalizat care loghează toate alocările și dealocările.
3. Scrie un eseu scurt despre avantajele RAII față de garbage collection.

Modulul 8. Programare generică

Obiective

- Înțelegerea abloanelor de funcții și clase.
- Specializarea parțială și totală.
- Introducere în conceptele (C++20) pentru constrângeri clare ale abloanelor.

Teorie

Abloanele permit scrierea de cod generic reutilizabil. Parametrii de ablon pot fi tipuri, valori sau abloane. 'template<typename T>' și 'template<class T>' sunt echivalente. Conceptele introduc predicate semantice care verifică proprietățile parametrilor.

Exemplu: ablon cu concept

```

#include <concepts>
#include <iostream>
#include <numeric>
#include <vector>

template <std::floating_point T>
T calculeaza_medie(const std::vector<T>& valori) {
    if (valori.empty()) {
        throw std::invalid_argument{"Nu se poate calcula media unui vector gol"};
    }
    T suma = std::accumulate(valori.begin(), valori.end(), T{0});
    return suma / static_cast<T>(valori.size());
}

int main() {
    std::vector<double> temperaturi{21.5, 23.0, 19.8};
    std::cout << calculeaza_medie(temperaturi) << '\n';
}

```


Exerciții

1. Creează o clasă șablon 'Matrice<T>' cu operații de bază și operatori suprascrise.
2. Scrie o funcție șablon 'swap_sigur' care funcționează cu orice tip mutabil și demonstrează folosind concepte.
3. Implementează un sistem de evenimente folosind șabloane și 'std::function'.

Modulul 9. Gestionarea erorilor și debugging

Obiective

- Folosirea mecanismului de excepții ('try', 'catch', 'throw').
- Tehnici de diagnosticare: 'assert', logare, debuggere.
- Scrierea testelor unitare de bază (framework 'Catch2' sau 'GoogleTest').

Teorie

Excepțiile trebuie folosite pentru condiții excepționale, nu pentru controlul normal al fluxului. Hierarhiile de excepții ar trebui să fie specifice contextului aplicației. Debugging-ul eficient implică reproducerea erorii, izolarea cauzei și verificarea soluției.

Exemplu: gestiune simplă a excepțiilor

```
#include <exception>
#include <iostream>

int divide(int a, int b) {
    if (b == 0) {
        throw std::runtime_error{"^ mprire la zero"};
    }
    return a / b;
}

int main() {
    try {
        std::cout << divide(10, 0) << '\n';
    } catch (const std::exception& ex) {
        std::cerr << "Eroare: " << ex.what() << '\n';
    }
}
```

Exerciții

1. Adaugă tratament de erori la funcțiile implementate în modulele anterioare.
2. Configurează un proiect mic cu 'Catch2' și scrie teste pentru o funcție 'fibonacci'.
3. Folosește un debugger (gdb, lldb sau Visual Studio) pentru a urmări execuția unui

algoritm de sortare.

Modulul 10. Funcționalități moderne (C++11 - C++20)

Obiective

- Inițializatori cu acolade, 'auto', 'decltype', 'nullptr'.
- Lambda expressions și 'std::function'.
- 'std::thread', 'std::async', 'std::future' pentru programare concurentă.
- 'ranges', 'structured bindings', 'std::optional', 'std::variant'.

Teorie

Standardele moderne au modernizat limbajul. C++11 a introdus mutarea semantică ('std::move'), punând bazele pentru performanțe și alocări suplimentare. C++17 a adus 'if constexpr' și 'std::optional', iar C++20 a continuat cu 'ranges', concepte și corutine. Adoptarea acestor caracteristici crește expresivitatea codului și reduce boilerplate-ul.

Exemplu: utilizarea 'std::optional' și a lambda-urilor

```
#include <iostream>
#include <optional>
#include <string>

std::optional<std::string> cauta_utilizator(int id) {
    if (id == 42) {
        return std::string{"Ada"};
    }
    return std::nullopt;
}

int main() {
    auto afiseaza = [](const std::optional<std::string>& utilizator) {
        if (utilizator) {
            std::cout << "Utilizator găsit: " << *utilizator << '\n';
        } else {
            std::cout << "Utilizator inexistent\n";
        }
    };

    afiseaza(cauta_utilizator(42));
    afiseaza(cauta_utilizator(7));
}
```

Exerciții

1. Rescrie proiectul din modulul 5 folosind 'std::unique_ptr', 'std::optional' și lambda-uri.
2. Creează un program care lansează mai multe fire de execuție pentru a calcula suma pătratelor dintr-un vector mare.
3. Utilizează 'std::ranges' pentru a filtra și transforma o listă de structuri personalizate.

Modulul 11. Proiect integrator

Tema: Gestionarea unui centru educațional

Cerințe

- Dezvoltă o aplicație de linie de comandă care permite gestionarea cursanților, cursurilor și profesorilor.
- Datele se stochează în fișiere JSON sau CSV.
- Aplicația trebuie să permită: adăugare, actualizare, căutare, listare și raportare.
- Folosește OOP, STL, vectori și mecanisme moderne ('std::optional', 'std::weak_ptr', etc.).

Etape recomandate

1. Modelează entitățile principale ('Student', 'Curs', 'Profesor') cu clase și relații adecvate.
2. Creează un modul de persistență care citește/scrie datele. Poți folosi biblioteci externe precum 'nlohmann::json' sau poți implementa un format simplificat.
3. Proiectează un meniu interactiv și fluxuri de lucru clare.
4. Scrie teste unitare pentru componentele critice (import/export, căutare).
5. Documentează aplicația și pregătește un ghid de utilizare.

Provocări suplimentare

- Extinde aplicația cu funcționalitate de planificare a orarului.
- Adaugă suport pentru export de rapoarte în format PDF sau HTML.
- Integrează autentificare și roluri (administrator, profesor, cursant).

Modulul 12. Resurse, bune practici și concluzii

Bune practici

- Respectă ghidurile de stil (Google C++ Style Guide sau LLVM Coding Standards).
- Automatizează build-ul cu CMake și configurează un sistem de integrare continuă.
- Scrie documentație cu Doxygen și menține un jurnal al deciziilor tehnice.

Resurse recomandate

- *C++ Primer* (Bjarne Stroustrup), *Effective Modern C++* (Scott Meyers), *C++ Tutorials*

â€¢ **Cursuri online:** Coursera â€¢ C++ For C Programmersâ€¢ , Udemy â€¢ Beginning C++ Programming
â€¢ **Situri & comunități:** cppreference.com, Stack Overflow \(\text{eticheta [c++]}\), ISO C++ Foundation

Strategii de învățare

1. Practică zilnică : rezolvă câteva exerciții în fiecare zi pentru a crea reflexe.
2. Citește cod sursă de calitate: biblioteci open-source precum Boost, fmt, absl.
3. Participă la comunități și la evenimente \(\text{meetup-uri locale, conferințe online}\).

Test final sugerat

1. Explică diferența dintre copiere și mutare în C++ și oferă exemple practice.
2. Proiectează un modul de logare thread-safe folosind 'std::mutex' și RAII.
3. Scrie o aplicație mică care utilizează thread-uri, STL și programare concurentă pentru a analiza un set de date.

Concluzii

Parcursul acestui curs ar trebui să îți ofere o imagine holistică asupra limbajului C++. Cheia este practica constantă și revizuirea periodică a conceptelor. Continuă să urmărești evoluția standardului și să experimentezi cu noi biblioteci și paradigme.

Anexă A. Soluții selectate

Modulul 2, exercițiul 1: convertor de temperatură

```
#include <iostream>

double celsius_in_fahrenheit(double celsius) {
    return celsius * 9.0 / 5.0 + 32.0;
}

double fahrenheit_in_celsius(double fahrenheit) {
    return (fahrenheit - 32.0) * 5.0 / 9.0;
}

int main() {
    double valoare{};
    char tip{};

    std::cout << "Introduceți valoarea temperaturii: ";
    std::cin >> valoare;
    std::cout << "Introduceți unitatea \(\text{C/F}\): ";
    std::cin >> tip;

    if (tip == 'C' || tip == 'c') {
        std::cout << valoare << " °C = " << celsius_in_fahrenheit(valoare) << " °F\n";
    }
```

```

    } else if (tip == 'F' || tip == 'f') {
        std::cout << valoare << " °F = " << fahrenheit_in_celsius(valoare) << " °C\n";
    } else {
        std::cout << "Unitate necunoscută .\n";
    }
}
}

```

Modulul 5, exercițiul 1: 'ContBancar'

```

#include <iostream>
#include <stdexcept>
#include <string>

class ContBancar {
public:
    ContBancar(std::string titular, double sold_initial)
        : titular_{std::move(titular)}, sold_{sold_initial} {}

    void depune(double suma) {
        if (suma <= 0) {
            throw std::invalid_argument{"Suma trebuie să fie pozitivă"};
        }
        sold_ += suma;
    }

    void retrage(double suma) {
        if (suma <= 0) {
            throw std::invalid_argument{"Suma trebuie să fie pozitivă"};
        }
        if (suma > sold_) {
            throw std::runtime_error{"Fonduri insuficiente"};
        }
        sold_ -= suma;
    }

    double sold() const noexcept { return sold_; }

    void afiseaza() const {
        std::cout << "Titular: " << titular_ << ", sold: " << sold_ << '\n';
    }

private:
    std::string titular_;
    double sold_;
};

int main() {
    ContBancar cont{"Ioana", 1000.0};
    cont.depune(250.0);
}

```

```

        cont.retrage\400.0\);
        cont.afiseaza\);
    }

```

Modulul 8, exercițiul 2: 'swap_sigur'

```

#include <concepts>
#include <iostream>
#include <utility>

template <typename T>
concept Mutabil = std::is_move_constructible_v<T> && std::is_move_assignable_v<T>;

template <Mutabil T>
void swap_sigur(T& a, T& b) noexcept(std::is_nothrow_move_constructible_v<T> &&) Tj T* (
    T tmp = std::move(a);
    a = std::move(b);
    b = std::move(tmp);
}

int main() {
    int x = 5, y = 9;
    swap_sigur(x, y);
    std::cout << x << ' ' << y << '\n';
}

```

Anexă B. Checklist de autoevaluare

- ☐ Pot explica diferența dintre o referință și un pointer.
- ☐ Îmi folosesc 'std::vector', 'std::map' și algoritmi STL de bază.
- ☐ Pot scrie și testa o clasă cu constructor, destructor și operatori.
- ☐ Am creat cel puțin un proiect complet folosind conceptele din curs.
- ☐ Urmezesc periodic noutățile standardului C++ și actualizez stilul de cod corespunzător.